¿Qué siente quien tuitea?

Clasificación de sentimiento en español con CNN 1D y redes recurrentes

Taller ABP – Entrega 1 · Redes Neuronales

Ingeniería en Analítica de Datos

[Nombre(s) del/los autor(es)] · [Docente]

Licencia CC BY 4.0

El problema

Más opiniones de las que un equipo puede leer

- Las organizaciones reciben miles de opiniones en redes sociales cada día.
- Objetivo: clasificar cada tweet como negativo, neutro o positivo.
- Valor: monitorear reputación y priorizar quejas sin leer todo a mano.
- Tipo de problema: clasificación multiclase de texto (datos no estructurados y secuenciales).

Ejemplos reales del dataset

“Esto de estar feliz mola” → positivo

“estoy hasta el ojete de que me digáis que tengo cara de mala leche” → negativo

“@user Por? Tenía pensado verla después de la segunda de Daredevil” → neutro

Los datos

InterTASS 2017 (UMSAB), subconjunto en español · fuente: cardiffnlp/xlm-t

1.839	324	870	3
tweets de entrenamiento	validación	prueba	clases balanceadas (613 c/u)

- Particiones oficiales train / val / test; la prueba solo se usa al final.
- Tweets cortos: 15,9 palabras en promedio, percentil 95 = 26 → secuencias de 40 tokens.
- Reto principal: conjunto pequeño → alto riesgo de sobreajuste.

Procesamiento de los datos

De texto crudo a tensores, sin fuga de información

1. Normalizar

minúsculas, URLs y @menciones a tokens, hashtags sin #, alargamientos reducidos

2. Conservar señal

signos ! ? ¡ ¿ y emojis, con carga emocional

3. Vectorizar

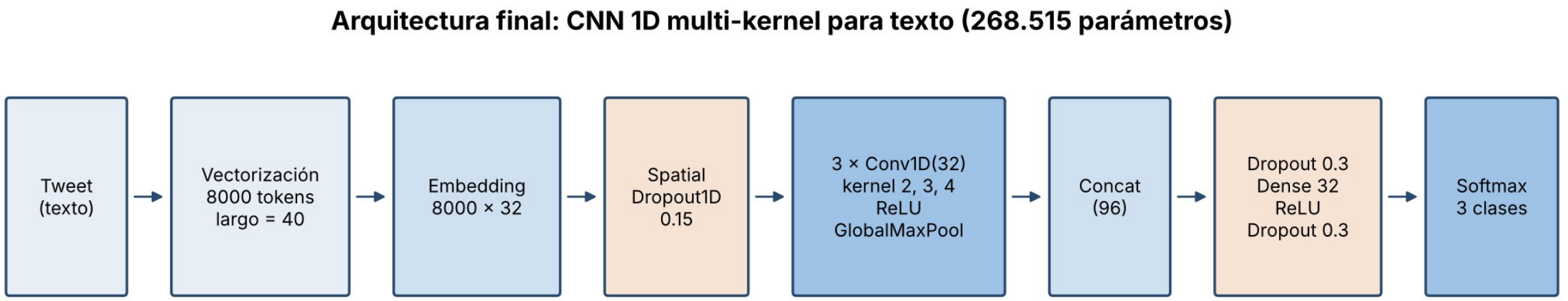
TextVectorization: 8.000 tokens, largo 40, ajustada solo con entrenamiento

4. Embedding

vector denso de 32 dimensiones por palabra, aprendido

Arquitectura de la red

CNN 1D multi-kernel · 268.515 parámetros

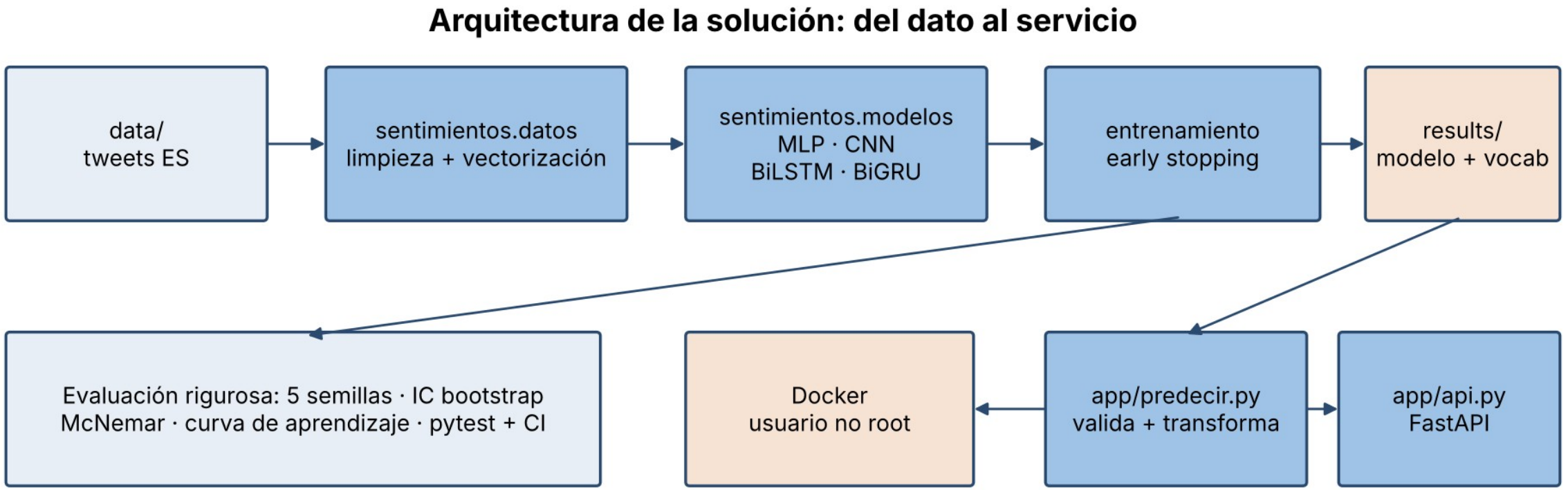


Cada kernel detecta n-gramas (2, 3 y 4 palabras); el max-pooling global conserva la señal más fuerte de cada filtro.

- Kernels de 2, 3 y 4 palabras detectan n-gramas; el max-pooling global conserva la señal más fuerte.
- Adam (lr 1e-3), entropía cruzada, batch 32 · Dropout, L2, early stopping y ReduceLROnPlateau.

Del dato al servicio

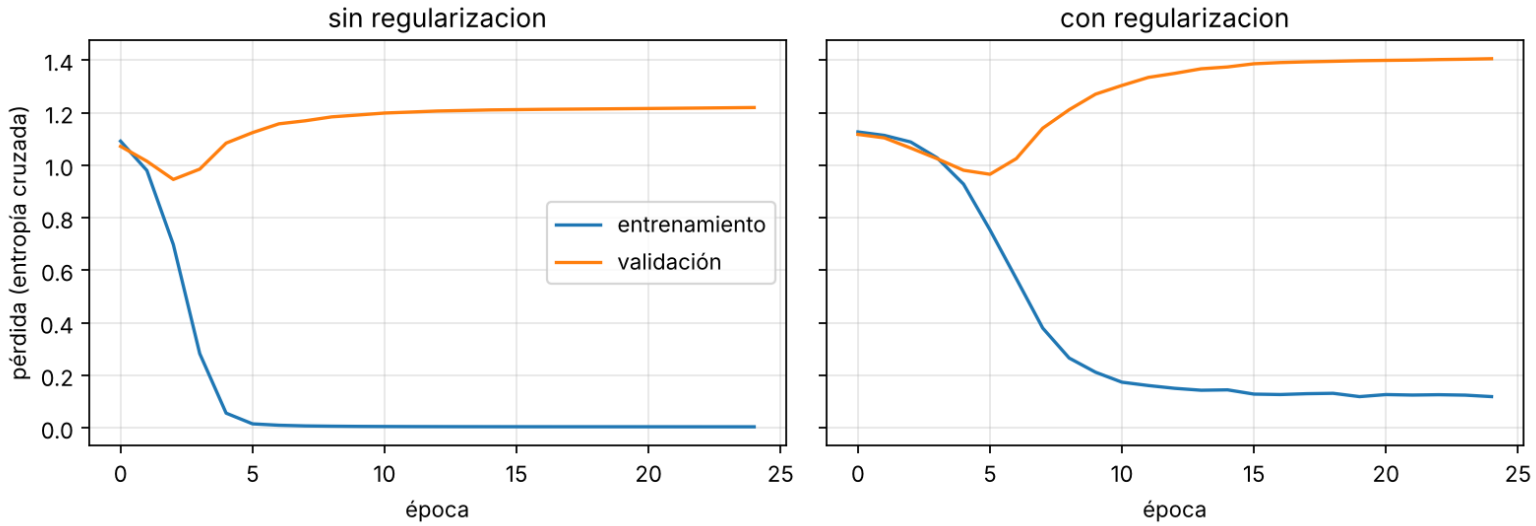
Arquitectura modular: cada pieza se puede probar y reemplazar



- Paquete src/sentimientos (datos · modelos · entrenamiento), app/ (CLI + API REST), 11 pruebas con pytest y CI en GitHub Actions.
- Seguridad: entradas validadas, vocabulario en JSON (sin pickle), contenedor sin privilegios, la API no guarda los textos.

El enemigo: el sobreajuste

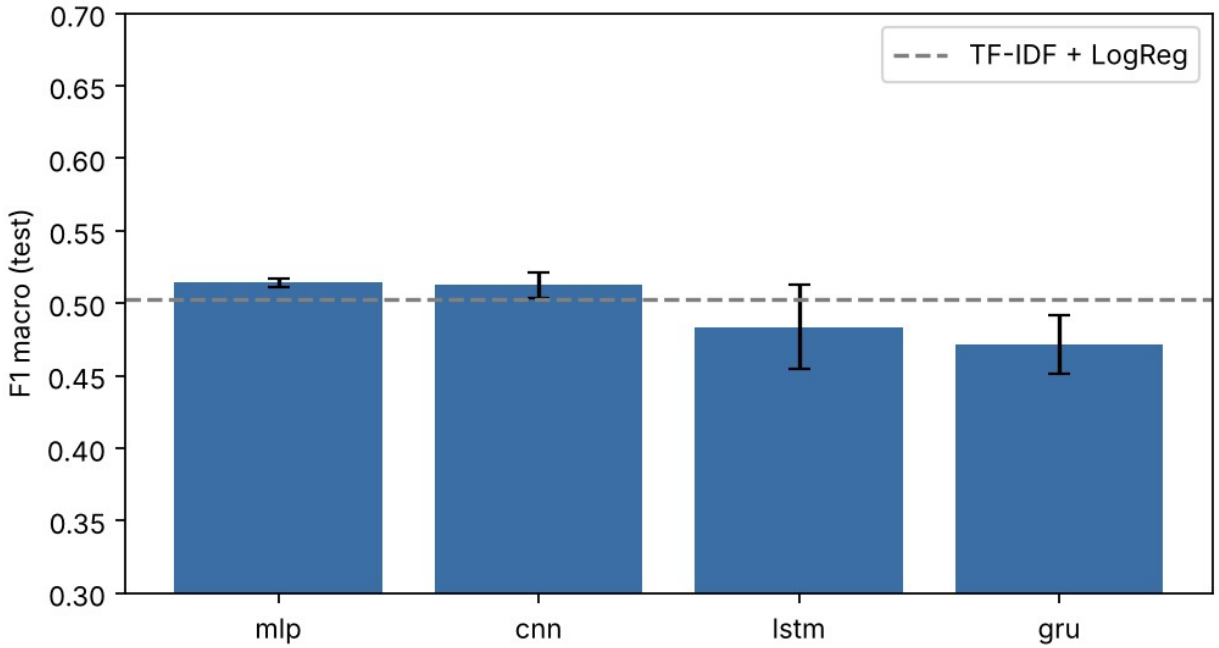
Ablación con y sin regularización (CNN, sin early stopping)



- Sin regularizar, la pérdida de entrenamiento llega a ~0 y la de validación sube desde la época 3.
- Dropout + L2 retrasan el sobreajuste, pero no lo eliminan.
- El early stopping es la defensa más efectiva.
- Mejora en prueba (F1 0,514 → 0,523): dentro del ruido.

Comparación de arquitecturas

Media de 3 semillas en el conjunto de prueba



Modelo	Exactitud	F1 macro
TF-IDF + LogReg	0,505	0,503
MLP	0,520	0,514
CNN 1D	0,514	0,512
BiLSTM	0,493	0,484
BiGRU	0,484	0,471

Con ~1.800 ejemplos, las RNN son más inestables (±2–3 puntos) y rinden menos que la CNN y el MLP.

Resultado final

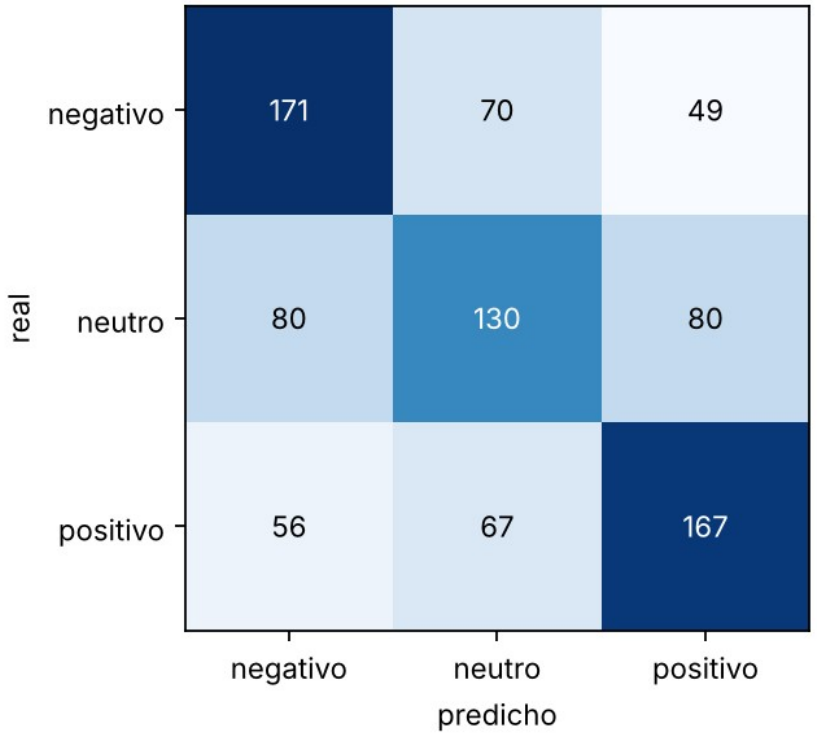
CNN afinada con búsqueda aleatoria (14 configuraciones × 3 semillas, elegida en validación)

51,6 %

exactitud, media de 5 semillas (±1,2)

53,8 %

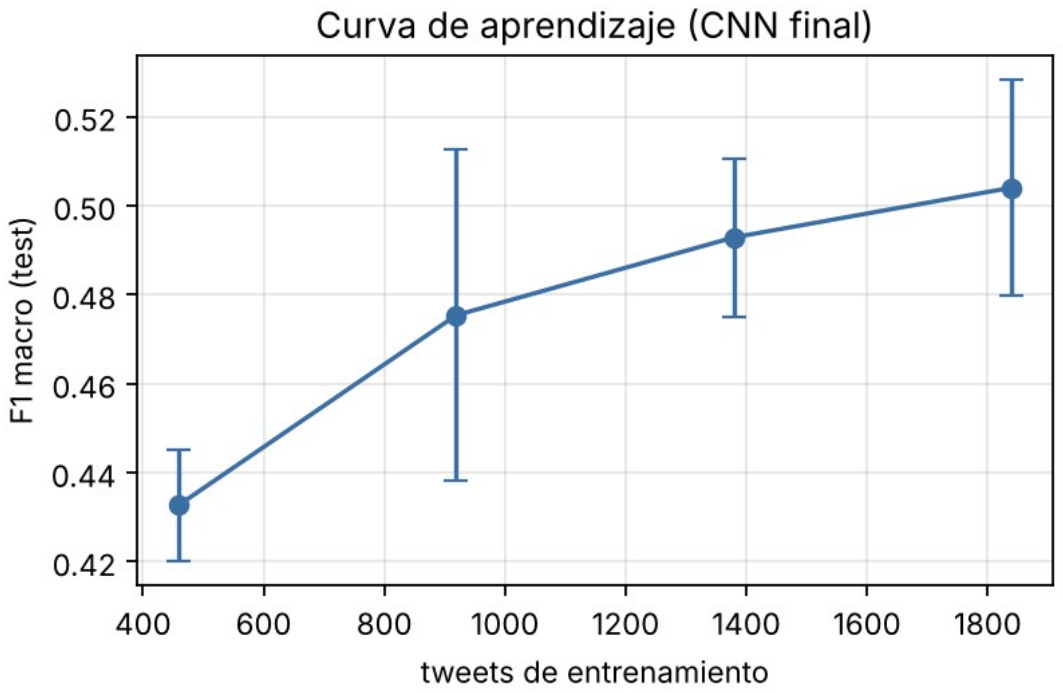
exactitud del ensamble de 5 modelos



- F1 por clase (ensamble): negativo 0,57 · neutro 0,47 · positivo 0,57.
- La clase neutra se confunde con ambas polaridades: ironía, lenguaje coloquial y ambigüedad.
- Lección metodológica: una búsqueda con una sola semilla eligió un LSTM que falló en prueba; se corrigió promediando semillas.

Rigor estadístico

¿Las diferencias son reales o ruido? (870 tweets de prueba)



Comparación	Prueba	p-valor
CNN vs. azar (33,3 %)	binomial	< 1e-34
CNN vs. BiGRU	McNemar	0,032
CNN vs. TF-IDF	McNemar	0,055
CNN vs. MLP	McNemar	0,63

- IC 95 % de la CNN (ensamble): 50,6–57,2 %.
- La CNN supera claramente a la BiGRU; frente a TF-IDF la evidencia es sugerente, no concluyente.
- La curva sigue subiendo: el límite son los datos, no la arquitectura.

Conclusiones y trabajo futuro

Lo que aprendimos

- Con pocos datos, la CNN 1D es más estable que LSTM/GRU.
- El sobreajuste domina; early stopping y evaluar con varias semillas son esenciales.
- La ganancia sobre TF-IDF + regresión logística es modesta (1–3 puntos): hay que reportarlo con honestidad.

Siguientes pasos

- Embeddings preentrenados en español (fastText, GloVe).
- Fine-tuning de BERT/XLM-R.
- Aumento de datos y validación cruzada estratificada.
- Intervalos de confianza para comparar modelos.

Prototipo y potencial de investigación

Prototipo funcional

- CLI: python app/predecir.py "texto"
- API REST: POST /predict · GET /health · documentación en /docs
- Docker sin privilegios; pruebas de caso válido, límite, entrada inválida y error
- Probada en vivo: "Qué asco de servicio" → negativo (0,66)

Líneas para el semillero

- Pocos datos en español: embeddings preentrenados (fastText, BETO) y fine-tuning de XLM-R.
- Ironía y clase neutra: análisis de errores sistemáticos.
- Robustez entre dialectos y auditoría de sesgos.
- Protocolo transferible: multi-semilla, IC, McNemar y curva de aprendizaje.

Repositorio y licencia

- Código: src/ y notebooks/proyecto_sentimientos.ipynb (reproducible, semillas fijas).
- Resultados: results/ (métricas, figuras, modelo, log).
- README con dataset, arquitectura, ejecución y métricas.
- Licencia: Creative Commons Atribución 4.0 (CC BY 4.0).

Repositorio público

github.com/<usuario>/<repositorio>

Referencias: Barbieri et al. (2022), XLM-T, LREC · Díaz Galiano et al. (2018), TASS 2017 · Goodfellow et al. (2016), Deep Learning · Berzal (2018).